



FACULTAD DE CIENCIAS EXACTAS, FÍSICAS y NATURALES

Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales

Unidad aritmético-lógica (ALU)

Trabajo práctico 1 - Arquitectura de computadoras

2026

Autores:

Krede, Julián

Piñera, Nicolás

Índice

1	Introducción	2
2	Especificación	2
3	Diseño	3
3.1	Arquitectura general	3
3.2	Módulo ALU	4
3.3	Módulo top	6
3.4	Asignación de pines	7
4	Verificación	7
4.1	Casos de prueba	8
4.2	Resultados	8
5	Síntesis e implementación	9
5.1	Utilización de recursos	9
5.2	Análisis de tiempos	10
6	Conclusiones	12

1 Introducción

La Unidad Aritmético-Lógica (ALU, por sus siglas en inglés Arithmetic Logic Unit) es uno de los componentes fundamentales de la arquitectura de cualquier procesador digital. Se trata del bloque combinacional (o secuencial, según el diseño) encargado de ejecutar las operaciones aritméticas y lógicas que constituyen la base del procesamiento de datos en un sistema digital, formando parte esencial de la unidad central de procesamiento (CPU) junto con la unidad de control y los registros.

Entre las operaciones típicas que realiza una ALU se encuentran las aritméticas —como la suma, la resta, la multiplicación y, en algunos casos, la división— y las lógicas, entre las que se destacan las operaciones AND, OR, XOR, NOT, así como operaciones de desplazamiento (shift) y rotación de bits. La selección de la operación a ejecutar se realiza generalmente mediante una señal de control u opcode, que indica a la ALU qué función debe aplicar sobre los operandos de entrada en un momento dado.

Desde el punto de vista del diseño digital, la ALU representa un caso de estudio particularmente relevante, ya que combina conceptos de lógica combinacional, diseño modular y optimización de recursos. Su implementación mediante lenguajes de descripción de hardware (HDL), como Verilog, permite modelar su comportamiento a distintos niveles de abstracción —comportamental, de flujo de datos o estructural— y posteriormente sintetizarla para su implementación física en dispositivos programables (FPGA) o circuitos integrados de aplicación específica (ASIC).

En este informe se describe el diseño de una ALU en Verilog y su implementación en la placa Basys 3. Primero se presenta la especificación planteada por el enunciado y luego el diseño de los módulos que componen la solución. A continuación se describe la verificación mediante simulación y, por último, se analizan los resultados de la síntesis e implementación en cuanto a utilización de recursos y tiempos.

2 Especificación

El enunciado plantea implementar en la placa Basys 3 una ALU con las siguientes características:

- El ancho del bus de datos debe ser **parametrizable**, para poder reutilizar el módulo en el trabajo final.
- Los operandos A y B y el código de operación se ingresan por medio de los *switches* de la placa, y se cargan en la ALU usando tres botones.
- El resultado de la operación se muestra en los LEDs.
- El diseño debe validarse mediante un *testbench* con generación de entradas aleatorias y chequeo automático, y simularse en Vivado incluyendo análisis de tiempo.

La Figura 1 muestra el esquema de la ALU planteado en el enunciado, y la Tabla 1 resume las operaciones que debe soportar junto con sus códigos. Estos códigos coinciden con el campo **funct** de las instrucciones tipo R de MIPS, lo que anticipa el uso de la ALU dentro del procesador del trabajo final.

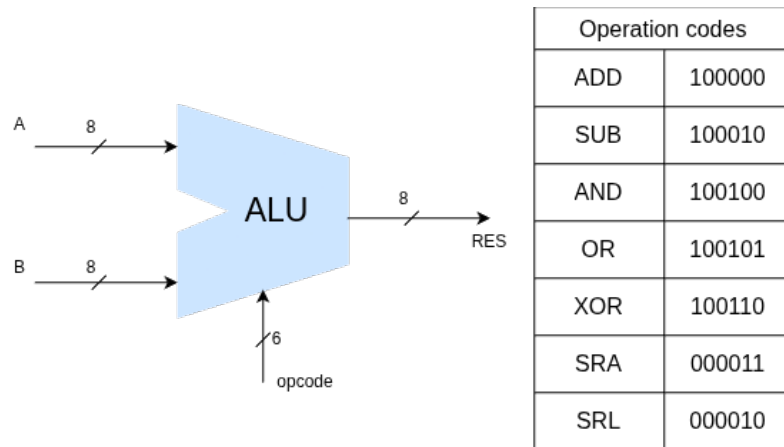


Figura 1: Esquema de la ALU planteado en el enunciado.

Cuadro 1: Operaciones soportadas por la ALU.

Operación	Código	Expresión	Descripción
ADD	100000	$A + B$	Suma con signo
SUB	100010	$A - B$	Resta con signo
AND	100100	$A \wedge B$	AND bit a bit
OR	100101	$A \vee B$	OR bit a bit
XOR	100110	$A \oplus B$	XOR bit a bit
SRA	000011	$A \ggg B$	Desplazamiento aritmético a derecha
SRL	000010	$A \gg B$	Desplazamiento lógico a derecha
NOR	100111	$\overline{A \vee B}$	NOR bit a bit

3 Diseño

3.1 Arquitectura general

El problema se dividió en dos partes bien diferenciadas:

- **La ALU** (`alu.v`): un bloque puramente **combinacional**, sin ningún conocimiento de la placa, que recibe dos operandos y un opcode y entrega el resultado junto con dos *flags*. Al no depender de la interfaz física, es el módulo que se reutilizará tal cual en el trabajo final.
- **El módulo top** (`top.v`): la parte **secuencial** que adapta la ALU a la placa. Contiene los registros donde se almacenan A, B y el opcode, decide cuál de ellos se carga a partir de las entradas de selección, y conecta la salida de la ALU a los LEDs.

Esta separación permite verificar la ALU de forma aislada y deja toda la lógica dependiente de la placa concentrada en un único módulo. La Figura 2 muestra el esquema resultante.

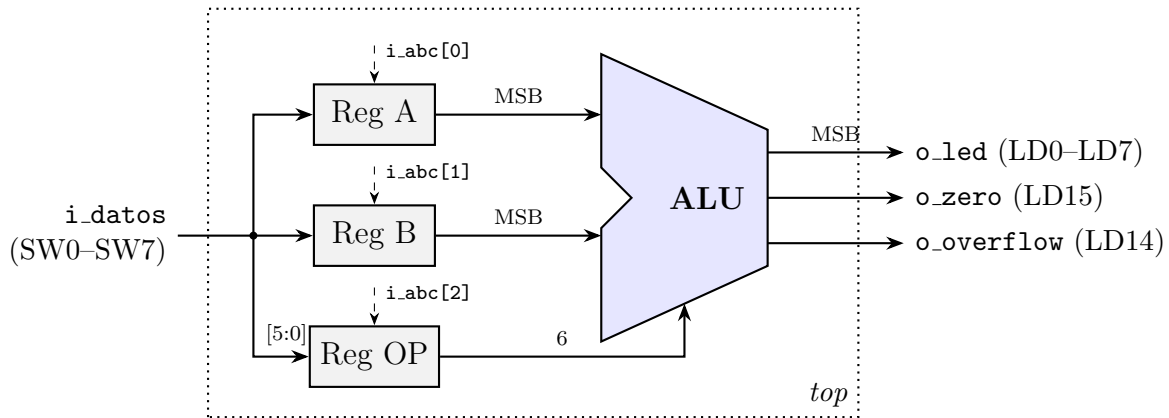


Figura 2: Diagrama en bloques del diseño implementado. Los registros se actualizan con el flanco ascendente de `clock` y se ponen a cero con `i_reset`.

3.2 Módulo ALU

Interfaz y parametrización

La ALU se describió como un módulo parametrizado por `MSB`, que define el ancho en bits de los operandos y del resultado (por defecto 8, que es la cantidad de *switches* y LEDs utilizados). El opcode es siempre de 6 bits, ya que no depende del ancho del dato.

Código 1: Declaración de la interfaz de la ALU.

```

1 module alu #(parameter MSB = 8)(
2     output reg      [MSB-1:0]    o_resultado,
3     output reg      o_zero,
4     output reg      o_overflow,
5     input signed    [MSB-1:0]    i_a,
6     input signed    [MSB-1:0]    i_b,
7     input           [5:0]        i_opc
8 );

```

Todos los anchos y constantes internas se expresan en función del parámetro (por ejemplo, el resultado por defecto es `{MSB{1'b0}}` y el bit de signo es `[MSB-1]`), de modo que basta con cambiar `MSB` al instanciar el módulo para obtener una ALU de otro ancho sin modificar el código.

Los operandos se declararon `signed` porque la ALU trabaja en complemento a dos: esto es necesario para que el desplazamiento aritmético replique el bit de signo y le da sentido a la detección de *overflow*.

Respecto a la especificación original (Figura 1), se agregaron dos salidas de estado, `o_zero` y `o_overflow`, pensando en su utilidad para el procesador del trabajo final (por ejemplo, para resolver saltos condicionales o detectar excepciones aritméticas).

Selección de la operación

La ALU se implementó con un único bloque `always@(*)` que contiene una sentencia `case` sobre el opcode. Cada rama calcula el resultado de una operación usando directamente los operadores de Verilog (+, -, &, |, ^, >>>, >>), delegando en la herramienta de síntesis la elección del hardware más adecuado (por ejemplo, las cadenas de acarreo `CARRY4` de la FPGA para la suma y la resta).

Un aspecto cuidado en esta descripción es evitar la inferencia de *latches*. En lógica combinacional, si alguna salida no se asigna en todos los caminos posibles, el sintetizador debe “recordar” su valor anterior e infiere un elemento de memoria no deseado. Para evitarlo:

- **o_overflow** se inicializa en 0 al comienzo del bloque, y solo se pone en 1 en las ramas de ADD y SUB.
- **o_resultado** se asigna en todas las ramas, incluida la rama **default**, que lo pone en cero ante un opcode no válido (dejando los LEDs apagados).
- **o_zero** se calcula siempre, al final del bloque, a partir del resultado.

Desplazamientos

Para los desplazamientos se tomó a A como el valor a desplazar y a B como la cantidad de posiciones. La diferencia entre ambos es cómo se completan los bits más significativos:

- **SRL (>>)**: se completa con ceros. Por ejemplo, $10010000 \gg 2 = 00100100$.
- **SRA (>>>)**: se replica el bit de signo, preservando el signo del número en complemento a dos. Por ejemplo, $10010000 \ggg 2 = 11100100$, es decir, $-112/4 = -28$.

El operador >>> solo realiza extensión de signo si el operando izquierdo es **signed**, razón por la cual fue necesario declarar así las entradas. La cantidad a desplazar, en cambio, Verilog siempre la interpreta sin signo, por lo que valores de B mayores o iguales a MSB desplazan todos los bits fuera del resultado.

Flags

Zero. Se activa cuando el resultado es nulo, independientemente de la operación. Hay que tener en cuenta que, como un opcode inválido produce un resultado nulo, en ese caso el *flag* también se activa (por ejemplo, inmediatamente después de un reset, cuando el opcode vale 000000).

Overflow. Solo tiene sentido en las operaciones aritméticas, por lo que se evalúa únicamente en ADD y SUB. Se detecta analizando los bits de signo, sin necesidad de calcular un bit extra de acarreo:

- **Suma:** hay overflow si A y B tienen el mismo signo y el resultado tiene signo distinto. Sumar dos números de signos opuestos nunca puede desbordar.
- **Resta:** como $A - B = A + (-B)$, hay overflow si A y B tienen signos *distintos* y el signo del resultado es distinto al de A.

Código 2: Detección de overflow en suma y resta.

```

1 // ADD
2 6'b100000: begin
3     o_resultado = i_a + i_b;
4     if (i_a[MSB-1] == i_b[MSB-1] && o_resultado[MSB-1] != i_a[MSB-1])
5         o_overflow = 1'b1;
6 end
7 // SUB
8 6'b100010: begin
9     o_resultado = i_a - i_b;
```

```

10  if (i_a[MSB-1] != i_b[MSB-1] && o_resultado[MSB-1] != i_a[MSB-1])
11      o_overflow = 1'b1;
12  end

```

Por ejemplo, con 8 bits: $100 + 50$ da como resultado 10010110 (-106), que es negativo a pesar de ser la suma de dos positivos, por lo que se activa el *flag*. En cambio, $100 - 50 = 50$ no produce overflow porque ambos operandos tienen el mismo signo.

3.3 Módulo top

Carga de operandos

Como la placa no dispone de suficientes *switches* para ingresar A, B y el opcode al mismo tiempo, se optó por compartir un único bus de entrada, *i_datos*, conectado a los *switches* SW0–SW7, y almacenar cada valor en un registro propio. La señal de 3 bits *i_abc* indica cuál de los tres registros se actualiza:

Cuadro 2: Selección del registro a cargar.

<i>i_abc</i>	Registro	Acción
001	A	$A \leq i_datos$
010	B	$B \leq i_datos$
100	opcode	$opcode \leq i_datos[5:0]$
otro	–	Todos los registros mantienen su valor

Se usó una codificación *one-hot*, en la que cada bit de *i_abc* corresponde a una entrada física distinta. Cualquier combinación con más de un bit activo cae en la rama **default** y no modifica ningún registro, lo que evita cargar por error el mismo dato en dos registros.

Los registros son sensibles al flanco ascendente del reloj de 100 MHz de la placa. Mientras la entrada de selección se mantiene activa, el registro correspondiente se carga en cada ciclo con el valor presente en los *switches*; al desactivarla, conserva el último valor. Para el opcode solo se toman los 6 bits menos significativos del bus.

Reset

Se implementó un reset **síncrono**, que se evalúa dentro del mismo bloque `always@(posedge clock)` y tiene prioridad sobre la carga. Al activarse lleva A, B y el opcode a cero, con lo cual la ALU queda en la rama **default**: los LEDs de resultado se apagan y se enciende el de *zero*.

Código 3: Lógica secuencial del módulo top.

```

1  always@(posedge clock) begin
2      if(i_reset) begin
3          A      <= {MSB{1'b0}};
4          B      <= {MSB{1'b0}};
5          opcode <= {6{1'b0}};
6      end
7      else begin
8          case(i_abc)
9              3'b001: A      <= i_datos;
10             3'b010: B      <= i_datos;

```

```

11      3'b100: opcode <= i_datos[5:0];
12      default: ; // Todos mantienen su valor
13  endcase
14  end
15 end

```

Conexión con la ALU

Los registros se conectan directamente a una instancia de la ALU, propagando el parámetro **MSB** (`alu #(MSB(MSB)) u_alu(...)`), y las salidas de la ALU se exponen sin registrar como salidas del módulo top. Por lo tanto, el resultado mostrado en los LEDs se actualiza en el mismo ciclo en que cambia alguno de los registros.

3.4 Asignación de pines

Para mapear los puertos del módulo top a la placa se partió del archivo de restricciones genérico de la Basys 3, descomentando y renombrando las líneas correspondientes. La asignación resultante se muestra en la Tabla 3.

Cuadro 3: Asignación de puertos a los recursos de la Basys 3.

Puerto	Recurso	Pines	Función
clock	Oscilador 100 MHz	W5	Reloj del sistema
i_datos[7:0]	SW7–SW0	W13, W14, V15, W15, W17, W16, V16, V17	Bus de datos
i_abc[0]	SW13	U1	Carga de A
i_abc[1]	SW14	T1	Carga de B
i_abc[2]	SW15	R2	Carga del opcode
i_reset	BTNC	U18	Reset
o_led[7:0]	LD7–LD0	V14, U14, U15, W18, V19, U19, E19, U16	Resultado
o_zero	LD15	L1	Flag zero
o_overflow	LD14	P1	Flag overflow

El reloj se declaró con `create_clock` con un período de 10 ns, lo que permite a Vivado realizar el análisis de tiempos sobre los caminos sincronizados por él. Las selecciones de carga se asignaron a los *switches* del extremo izquierdo en lugar de a botones, de modo que su estado quede visible y no sea necesario mantener un botón presionado.

4 Verificación

Para validar el diseño se escribió el testbench `tb_top.v`, que ejercita el módulo top completo (registros y ALU) de la misma forma en que se usaría la placa. Genera un reloj de 10 ns (igual al de la Basys 3), aplica un reset inicial y luego, para cada caso, carga A, B y el opcode en `i_datos`, activando durante un ciclo el bit de `i_abc` correspondiente a cada uno:

Código 4: Carga de un caso en el testbench.

```

1 // Caso feliz: 20 + 22 = 42
2 @(negedge clock); i_datos = 8'd20; i_abc = 3'b001;
3 @(negedge clock); i_datos = 8'd22; i_abc = 3'b010;
4 @(negedge clock); i_datos = ADD; i_abc = 3'b100;
5 @(negedge clock); i_abc = 3'b000;
6 #100;

```

Las entradas se modifican en el flanco de bajada del reloj para que estén estables cuando el módulo top las registra en el flanco de subida. Si se cambiaran en el mismo flanco de subida, el resultado dependería del orden en que el simulador evalúe los bloques (una condición de carrera). Entre caso y caso se dejan 100 ns para poder observar la salida en las formas de onda.

4.1 Casos de prueba

Además de un caso feliz, se eligieron casos extremos para cada operación: sumas y restas en los límites del rango representable en complemento a dos (donde aparece el *overflow*), resultados nulos, operaciones lógicas con todos unos y todos ceros, desplazamientos máximos y un opcode inválido. Por último, se aplica un reset con los registros cargados para verificar que la salida vuelve a cero. La Tabla 4 resume los casos y los valores esperados, que se contrastaron con las formas de onda obtenidas.

Cuadro 4: Casos de prueba del testbench y resultados esperados.

Op.	A	B	Caso	Resultado	Z	OV
ADD	20	22	Caso feliz	42	0	0
ADD	127	1	Máximo positivo +1	-128	0	1
ADD	-128	-1	Mínimo negativo -1	127	0	1
ADD	127	-127	Opuestos	0	1	0
SUB	-128	1	Mínimo negativo -1	127	0	1
SUB	127	-1	Máximo positivo +1	-128	0	1
SUB	0	-128	+128 no representable	-128	0	1
SUB	-128	-128	Iguales	0	1	0
AND	FF	00	Todos ceros	00	1	0
OR	F0	0F	Complementarios	FF	0	0
XOR	FF	FF	Iguales	00	1	0
NOR	00	00	Todos ceros	FF	0	0
SRA	80	7	Desplazamiento máximo	FF	0	0
SRL	80	7	Desplazamiento máximo	01	0	0
SRL	FF	8	Fuera de rango	00	1	0
111111	55	66	Opcode inválido	0	1	0

Los casos de SRA y SRL usan el mismo operando, 10000000 (-128), para poner en evidencia la diferencia entre ambos: el desplazamiento aritmético replica el bit de signo y da -1, mientras que el lógico completa con ceros y da 1.

4.2 Resultados

El testbench se ejecutó en simulación comportamental y en simulación temporal post-implementación. Esta última utiliza la netlist ruteada, con los retardos reales de cada celda y conexión. En ambas simu-

laciones las salidas coincidieron con los valores de la Tabla 4. En la simulación post-implementación se observa además que la salida no cambia inmediatamente después del flanco de reloj, sino que tarda varios nanosegundos en estabilizarse, en concordancia con el análisis de tiempos de la Sección 5.2.

La Figura 3 muestra un fragmento de la simulación comportamental, entre los 880 y los 1060 ns aproximadamente, con los valores en hexadecimal. Se observan dos de los casos extremos de la resta:

- $127 - (-1)$: se carga $A = 7f$ ($i_abc = 1$) y luego $B = ff$ ($i_abc = 2$). Como el opcode sigue siendo SUB (22) del caso anterior, la salida cambia en cuanto se carga cada operando. Con $A = 127$ y el B anterior (1) se obtiene $7e$ (126) sin *overflow*. Al cargar $B = -1$, el resultado pasa a 80 (-128) y se activa $o_overflow$, ya que $+128$ no es representable en 8 bits. Por último se vuelve a cargar el opcode ($i_abc = 4$), sin cambios en la salida.
- $0 - (-128)$: al cargar $A = 00$, la salida pasa a 01 ($0 - (-1)$) y el *overflow* se desactiva. Al cargar $B = 80$ (-128), el resultado vuelve a ser 80 con *overflow*, ya que nuevamente el resultado exacto sería $+128$.

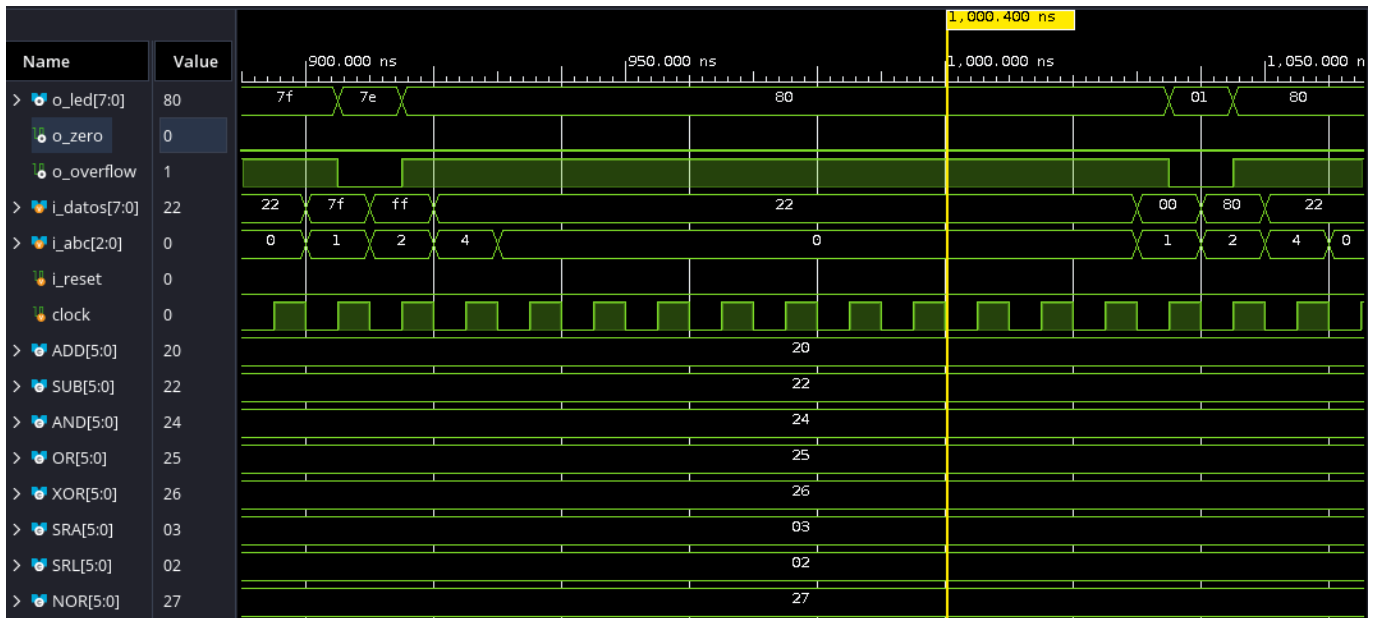


Figura 3: Formas de onda de la simulación comportamental: casos $127 - (-1)$ y $0 - (-128)$.

Esta captura también pone en evidencia el comportamiento del módulo top: como la ALU es combinacional y está conectada directamente a los registros, la salida refleja en todo momento la operación entre los valores cargados hasta ese instante, y se actualiza en el mismo flanco de reloj en que se carga cada registro.

5 Síntesis e implementación

El diseño se sintetizó e implementó con Vivado para la FPGA Artix-7 (XC7A35T) de la Basys 3.

5.1 Utilización de recursos

La Tabla 5 resume los recursos utilizados luego de la implementación, y la Figura 4 muestra el reporte jerárquico obtenido en Vivado, discriminado por módulo.

Cuadro 5: Utilización de recursos post-implementación.

Recurso	top (total)	u_alu	Disponible	Utilización
Slice LUTs	77	74	20800	0,37 %
Slice Registers	22	0	41600	0,05 %
F7 Muxes	11	11	16300	0,07 %
Slices	23	22	8150	0,28 %
Bonded IOB	23	0	106	21,70 %
BUFGCTRL	1	0	32	3,13 %

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
▼ N top	77	22	11	23	77	23	1
🔍 u_alu (alu)	74	0	11	22	74	0	0

Figura 4: Reporte de utilización jerárquico en Vivado.

Del reporte se desprenden algunas observaciones:

- La cantidad de registros es exactamente 22, que coincide con los bits de A, B y opcode ($2 \cdot \text{MSB} + 6 = 22$ para 8 bits). Todos pertenecen al módulo top: la ALU no utiliza ningún registro, lo que confirma que se sintetizó como lógica puramente combinacional y que no se infirieron *latches*.
- Prácticamente toda la lógica (74 de las 77 LUTs) corresponde a la ALU. Las 3 LUTs restantes pertenecen al módulo top y corresponden a la lógica de carga de los registros (decodificación de *i_abc* y *reset*).
- Los 11 multiplexores F7 son utilizados por la ALU para combinar las salidas de LUTs, lo que es consistente con la selección del resultado entre las distintas operaciones y con los desplazadores, que son esencialmente redes de multiplexores.
- Los 23 IOB corresponden a los puertos del top: 8 de *i_datos*, 3 de *i_abc*, *i_reset*, *clock*, 8 de *o_led*, *o_zero* y *o_overflow*. El único BUFG es el buffer global del reloj.

En conjunto, el diseño ocupa una fracción ínfima de la FPGA, lo que deja amplio margen para integrarlo en el procesador del trabajo final.

5.2 Análisis de tiempos

La Figura 5 muestra el *Timing Summary* obtenido luego de la implementación.

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	inf	Worst Hold Slack (WHS)	inf	Worst Pulse Width Slack (WPWS)	4.500 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	76	Total Number of Endpoints	76	Total Number of Endpoints	23

Figura 5: Resumen de tiempos post-implementación.

Tanto el WNS como el WHS resultan infinitos y no hay ningún *endpoint* que falle. Esto no significa que el diseño tenga un margen ilimitado, sino que **no existe ningún camino restringido por el reloj**:

- Los caminos desde los *switches* hasta los registros A, B y opcode parten de puertos de entrada sin `set_input_delay`.
- Como las salidas de la ALU no se registran, los caminos que la atraviesan parten de los registros y terminan directamente en los pines de los LEDs, que no tienen `set_output_delay`.

Al no haber caminos registro a registro, Vivado no tiene contra qué comparar los retardos, y los 76 *endpoints* quedan sin restricción.

El único parámetro con un valor finito es el *Worst Pulse Width Slack* (WPWS) de 4,5 ns, que verifica que los pulsos alto y bajo del reloj de 100 MHz (5 ns cada uno) sean más largos que el mínimo exigido por los *flip-flops*. Con este margen, el reloj de la placa no representa ninguna limitación.

Si bien estos caminos no se comparan contra el período del reloj, Vivado igualmente calcula su retardo. Para obtenerlo se analizaron los caminos que van desde los registros A, B y opcode hasta las salidas.

El camino más largo, en la esquina de proceso lenta, va desde `A_reg[2]` hasta `o_zero`, con un retardo de datos de 15,149 ns (5,766 ns de lógica y 9,382 ns de ruteo) y 8 niveles lógicos. La Tabla 6 muestra su desglose.

Cuadro 6: Desglose del camino crítico `A_reg[2] → o_zero`. Los tiempos son acumulados desde el flanco de reloj en el pin W5.

Etapas	Recursos	Tiempo (ns)
Distribución del reloj hasta el registro	IBUF, BUFG	5,154
Salida del registro A	FDRE	5,610
Sumador/restador	LUT2, 2 × CARRY4	7,600
Selección del resultado según el opcode	LUT5, MUXF7, LUT6	10,188
Detección de cero	LUT6	11,294
Ruteo hasta el pin L1	—	16,781
Buffer de salida	OBUF	20,303

El camino es coherente con la estructura del diseño: atraviesa la suma (implementada con las cadenas de acarreo `CARRY4`), el multiplexor que elige el resultado según el opcode, y finalmente la compuerta que detecta si el resultado es cero, que necesita conocer todos los bits del resultado y por eso es la salida más lenta. Los caminos siguientes son los de `o_overflow` (11,413 ns) y `o_led[5]` (10,630 ns).

Se observa además que la lógica de la ALU propiamente dicha, desde la salida del registro hasta la detección de cero, demora unos 5,7 ns. El resto del retardo corresponde a la distribución del reloj, al ruteo hasta el pin (5,5 ns, ya que el LED LD15 está alejado de la lógica) y al buffer de salida (3,5 ns).

Si en el procesador del trabajo final se necesita garantizar que la ALU opere dentro de un ciclo de reloj, bastará con registrar su salida (o las entradas de la etapa siguiente). De esa forma el camino pasa a ser registro a registro y el análisis de tiempos arroja un WNS real. Con unos 5,7 ns de lógica, es esperable que la ALU entre holgadamente en el período de 10 ns, aunque esto deberá confirmarse con el análisis de tiempos del diseño final, ya que el ruteo dependerá de la ubicación de la lógica.

6 Conclusiones

Se diseñó e implementó en la Basys 3 una ALU parametrizable que soporta las ocho operaciones pedidas, junto con los *flags* de zero y *overflow*. Separar la ALU, puramente combinacional, del módulo top que la adapta a la placa permitió mantenerla independiente del hardware, de modo que puede reutilizarse sin cambios en el trabajo final.

La simulación, tanto comportamental como post-implementación, confirmó el funcionamiento esperado en los casos extremos de cada operación, en particular en los límites del rango en complemento a dos, donde aparece el *overflow*.

La síntesis mostró que el diseño ocupa una fracción mínima de la FPGA (77 LUTs y 22 registros), y que no se infirieron *latches*. El análisis de tiempos indicó que el camino crítico atraviesa el sumador y la detección de cero. Como la salida de la ALU no está registrada, ese camino no queda acotado por el reloj; al integrarla en el procesador convendrá registrarla, para poder verificar que opere dentro de un ciclo.